

TASK10: #51614

GOAL: configure a EBS such that, it can be shared between different instances.

1. Create and set up an EBS such that it should be "Multi-Attach" enabled.
2. verify the working
3. study about Elastic File System in AWS.
4. configure an EFS shared between different ec2 instances.

Part 1: EBS Single-Attach Setup.

Steps performed

- Launched an EC2 instance (Instance-A) in a chosen Availability Zone.
- Created a new EBS volume (e.g. gp3), sized appropriately, in the same Availability Zone as the instance.
- Attached the volume to Instance-A via Actions → Attach Volume, specifying a device name (e.g. /dev/sdb).
- Formatted and mounted the volume on Instance-A, then wrote a test file to it.

Part 2: Verifying the EBS Configuration

On Instance-A, the attached block device was confirmed using:

```
lsblk
```

The volume was then formatted, mounted, and a test file written:

```
sudo mkfs.ext4 /dev/xvdf
sudo mkdir -p /mnt/data
sudo mount /dev/xvdf /mnt/data
echo "hello from Instance-A" | sudo tee /mnt/data/test.txt
```

Verifying data persistence by moving the volume

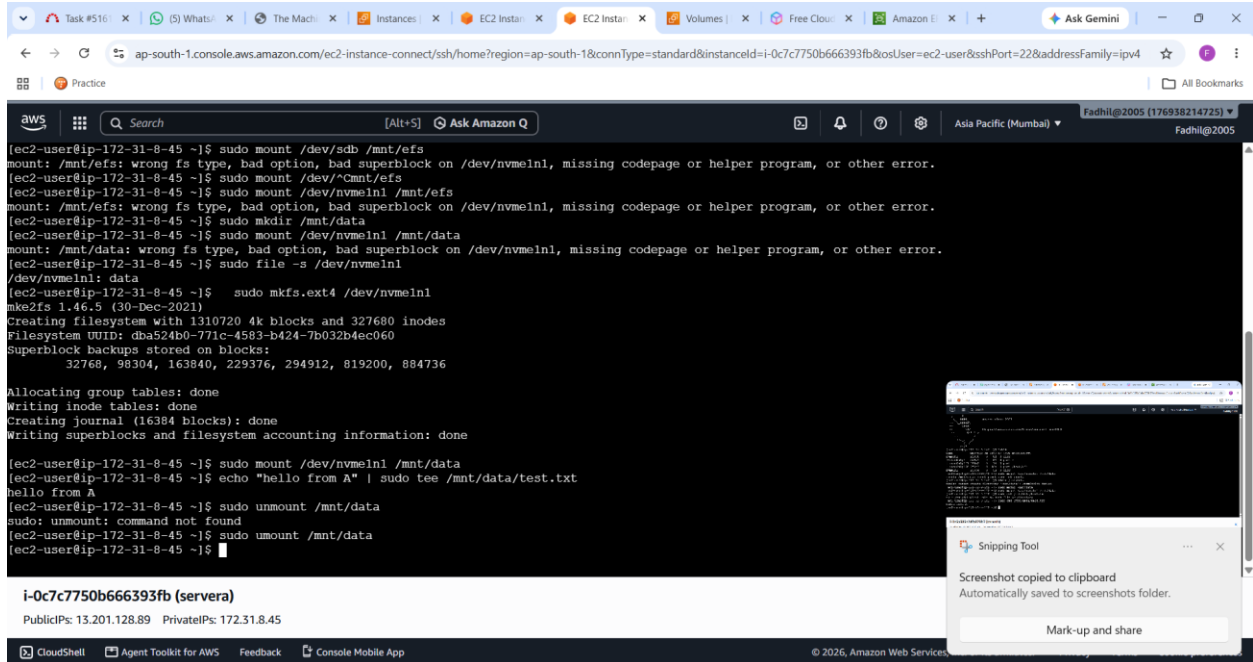
Since a standard EBS volume can only be attached to one instance at a time, sharing it between instances is demonstrated by detaching it from one instance and reattaching it to another (sequential access, not simultaneous).

- Unmounted and detached the volume from Instance-A (Volumes → Actions → Detach Volume).
- Attached the same volume to Instance-B (same Availability Zone required).
- Mounted the volume on Instance-B and confirmed the earlier test file was present:

```
sudo mkdir -p /mnt/data
sudo mount /dev/xvdf /mnt/data
cat /mnt/data/test.txt
```

This confirmed that the volume and its data can be moved between instances, but only one instance can access the volume at any given time. Standard EBS volumes are not designed for concurrent, simultaneous access by multiple instances — for that, Amazon EFS is the appropriate solution, as covered next.

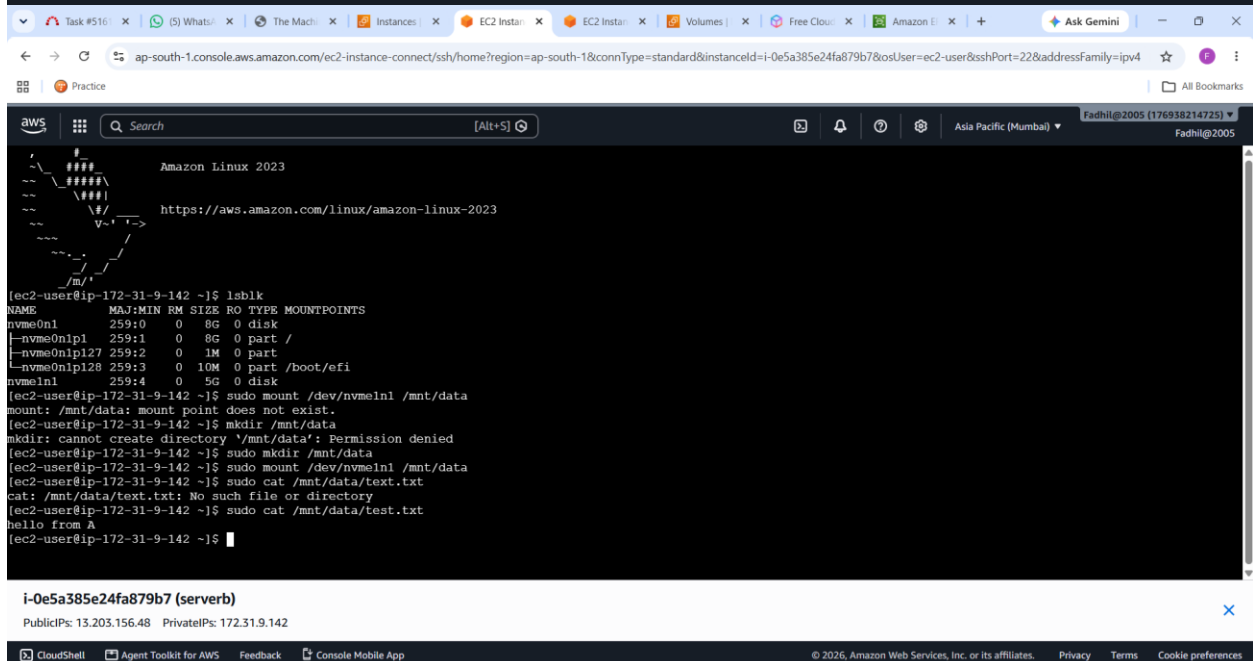
OUTPUTS:



The screenshot shows the AWS CloudShell interface for instance `i-0c7c7750b666393fb` (servera). The terminal output shows the user attempting to mount `/dev/sdb` and `/dev/nvme1n1` as `/mnt/efs` and `/mnt/data` respectively. Both attempts fail with the error: "wrong fs type, bad option, bad superblock on /dev/nvme1n1, missing codepage or helper program, or other error." The user then creates a directory `/mnt/data` and mounts `/dev/nvme1n1` to it. They also create a file `/dev/nvme1n1: data` and run `mkfs.ext4 /dev/nvme1n1`. The output shows the creation of a filesystem with 1310720 4k blocks and 327680 inodes. The user then mounts `/dev/nvme1n1` to `/mnt/data` and runs `echo "hello from A" | sudo tee /mnt/data/test.txt`. The output shows "hello from A". The user then unmounts `/mnt/data` and runs `sudo umount /mnt/data`. The output shows "sudo: umount: command not found". The user then runs `sudo umount /mnt/data` and the output shows "sudo: umount: command not found".

i-0c7c7750b666393fb (servera)
PublicIPs: 13.201.128.89 PrivateIPs: 172.31.8.45

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences



The screenshot shows the AWS CloudShell interface for instance `i-0e5a385e24fa879b7` (serverb). The terminal output shows the user running `lsblk` and getting the following output:

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINTS
nvme0n1	259:0	0	8G	0	disk	
nvme0n1p1	259:1	0	8G	0	part	/
nvme0n1p2	259:2	0	1M	0	part	
nvme0n1p3	259:3	0	10M	0	part	/boot/efi
nvme1n1	259:4	0	5G	0	disk	

The user then runs `sudo mount /dev/nvme1n1 /mnt/data` and the output shows "mount: /mnt/data: mount point does not exist." The user then runs `mkdir /mnt/data` and the output shows "mkdir: cannot create directory '/mnt/data': Permission denied". The user then runs `sudo mkdir /mnt/data` and the output shows "mkdir: cannot create directory '/mnt/data': Permission denied". The user then runs `sudo mount /dev/nvme1n1 /mnt/data` and the output shows "mount: /mnt/data: mount point does not exist." The user then runs `cat /mnt/data/text.txt` and the output shows "cat: /mnt/data/text.txt: No such file or directory". The user then runs `cat /mnt/data/test.txt` and the output shows "hello from A".

i-0e5a385e24fa879b7 (serverb)
PublicIPs: 13.203.156.48 PrivateIPs: 172.31.9.142

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Part 3: Study — Amazon Elastic File System (EFS)

What EFS is

Amazon EFS is a fully managed, elastic Network File System (NFS) that can be mounted concurrently by many EC2 instances (and other compute services) across multiple Availability Zones within a VPC.

EBS vs. EFS

Aspect	EBS (Single Attach)	EFS
Storage type	Block storage	File storage (NFS)
Attach scope	Single AZ; one instance at a time	Multi-AZ; thousands of clients
Concurrent access	Not supported — only one instance at a time	Natively safe for concurrent access
Scaling	Manual resize	Elastic, automatic
Typical use case	Clustered DBs, boot volumes	Shared content, home dirs, CMS uploads

Key concepts

- Performance modes: General Purpose (default, low latency) vs. Max I/O (higher throughput, for highly parallelized workloads).
- Throughput modes: Bursting, Provisioned, and Elastic.
- Storage classes: Standard, Standard-IA, One Zone, and One Zone-IA, with lifecycle management for automatic tiering.
- Mount targets: an ENI placed in each subnet/AZ that needs access; EC2 instances connect to EFS over NFS (port 2049) via these mount targets.

Part 4: Configuring EFS Shared Between EC2 Instances

Create the file system

- EFS Console → Create file system → Customize.
- Selected the same VPC used by the EC2 instances.
- Under Network settings, selected a subnet for each Availability Zone to create mount targets.

Mount targets created

The file system fs-01cbe4e4df2b2a428 has mount targets available in all three Availability Zones of the VPC (vpc-068e85fd115522277):

Availability Zone	Subnet ID	Mount Target State
ap-south-1a	subnet-0949b2de5b988c379	Available
ap-south-1b	subnet-0d2c2def1e7f5e675	Available
ap-south-1c	subnet-0a3613dd55208b291	Available

Security group configuration

The mount targets use security group sg-0991e1dff110d7610 (launch-wizard-1). An inbound rule was required:

- Type: NFS
- Port: 2049
- Source: the security group used by the EC2 instances that need to mount the file system

Mounting EFS on EC2 instances

On each instance (Amazon Linux):

```
sudo mkdir -p /mnt/efs
sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsz=1048576,hard,timeo=600,retrans=2,noresvport 172.31.33.221:/ efs
cd /mnt/efs
touch fi
```

Alternatively, when launching a new instance, the same VPC/subnet (matching an existing mount target) can be selected, and the existing EFS file system attached directly from the "File systems" section of the EC2 launch wizard — EC2 then automatically installs amazon-efs-utils, mounts the file system, and updates /etc/fstab---

Verifying shared access across instances

On Instance-A:

```
echo "hello from A" | sudo tee /mnt/efs/test.txt
```

On Instance-B:

```
cat /mnt/efs/test.txt
```

The file written from Instance-A was immediately visible and readable from Instance-B, confirming genuine concurrent shared access across multiple EC2 instances.

OUTPUT:

The screenshot displays two terminal windows from the AWS Management Console, showing the execution of commands on two different EC2 instances to verify shared access to an Amazon EFS file system.

Instance-A (top terminal):

```
[ec2-user@ip-172-31-8-45 ~]$ mkdir serverb
[ec2-user@ip-172-31-8-45 ~]$ sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsz=1048576,hard,timeo=600,retrans=2,noresvport 172.31.33.221:/ serverb
[ec2-user@ip-172-31-8-45 ~]$ cd serverb
[ec2-user@ip-172-31-8-45 serverb]$ ls
servera
[ec2-user@ip-172-31-8-45 serverb]$ touch serverb
touch: cannot touch 'serverb': Permission denied
[ec2-user@ip-172-31-8-45 serverb]$ sudo touch serverb
[ec2-user@ip-172-31-8-45 serverb]$ ls
servera serverb
[ec2-user@ip-172-31-8-45 serverb]$
```

Instance-B (bottom terminal):

```
[ec2-user@ip-172-31-9-142 ~]$ mkdir servera
[ec2-user@ip-172-31-9-142 ~]$ sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsz=1048576,hard,timeo=600,retrans=2,noresvport 172.31.33.221:/ servera
[ec2-user@ip-172-31-9-142 ~]$ cd servera
[ec2-user@ip-172-31-9-142 servera]$ touch servera
touch: cannot touch 'servera': Permission denied
[ec2-user@ip-172-31-9-142 servera]$ sudo touch servera
[ec2-user@ip-172-31-9-142 servera]$ ls
servera serverb
[ec2-user@ip-172-31-9-142 servera]$
```